

Actividad evaluable UT4A1 – Documentación de aplicaciones

**Yedra Sánchez Santana
2ºA DAM**



ÍNDICE

Creación de una nueva rama.....	3
Conexión del login con la base de datos.....	3
Creación del menú común a todas las páginas.....	4
Cómo evitar navegar a /home sin estar logueado: hook useEffect().....	5
Creación del componente que usarás en Home.tsx.....	6
Creación del formulario en Dashboard.tsx.....	6
Creación del fichero backend/services/items.js.....	10
Añadir funcionalidad al botón Insertar del formulario.....	11
Creación del resto de los endpoints: /getItems y /deleteItem.....	13
Obtener datos de la base de datos (/getItems).....	13
Borrar un registro de la base de datos (/deleteItem).....	16
Creación de la tabla en el fichero Dashboard.tsx: lógica para mostrar los datos en la tabla y lógica para borrar un registro de la tabla.....	18
Lógica para mostrar los datos en la tabla.....	18
Implementación de la tabla: componente Table.....	20
Lógica para borrar un registro de la tabla.....	21
Ejecución del programa:.....	22
Actividad evaluable UT4A1 – Documentación de aplicaciones.....	25
1. Rama del proyecto.....	25
2. Creación de ayuda contextual: componente Tooltip.....	25
3. Agregar el manual de usuario a tu aplicación.....	29
Enlace a GitHub.....	30

1. Creación de una nueva rama

Creamos una nueva rama en nuestro proyecto: nos situamos en la carpeta ProyectoYSS y ahí escribimos el siguiente comando:

git checkout -b crud

Con esto estamos creando una rama nueva llamada “crud” y además, nos estamos cambiando a dicha rama. Comprobamos que estamos en la nueva rama con el comando:

git branch

```
PS D:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> git checkout -b crud
fatal: a branch named 'crud' already exists
PS D:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> git branch
* crud
  login
  main
  redux
```

2. Conexión del login con la base de datos

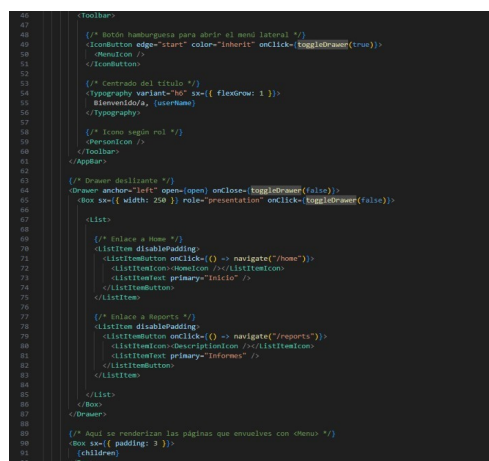
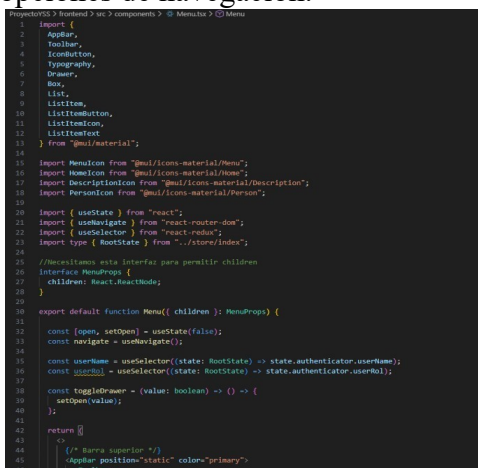
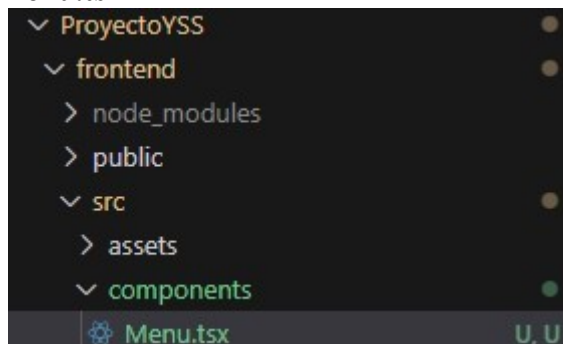
Con lo que se explicó en clase sobre el backend y los endpoints, reemplazamos la comprobación de usuario y login correcto con la llamada al endpoint /login para conectarnos a la base de datos y hacer la autenticación del usuario.

Implementamos la conexión del login con la base de datos real, modificando completamente la función handleSubmit. Transformamos en una función asíncrona que realiza una petición HTTP POST al endpoint '/api/login' del backend, enviando las credenciales del usuario en formato JSON para su validación en la base de datos MySQL.

Así queda nuestro login.tsx tras modificar el handleSubmit o como yo lo llamo handleLogin:

```
JS login.js  Login.tsx M
ProyectoYSS > frontend > src > pages > Login.tsx > Login
8 export default function Login() {
16 //FUNCIÓN ASÍNCRONA que conecta con el BACKEND
17 const handleLogin = async () => {
18   try {
19     //Realizamos la petición POST al backend enviando usuario y contraseña
20     const response = await fetch("/api/login", {
21       method: "POST",
22       headers: { "Content-Type": "application/json" },
23       body: JSON.stringify({ user, password }) //Datos enviados al backend
24     });
25
26     const data = await response.json();
27
28     //El backend devuelve data.data, si existe ese usuario en MySQL
29     if (!data.data || data.data.length === 0) {
30       alert("Usuario y/o contraseña incorrectos");
31       return;
32     }
33
34     //Recibimos del backend: nombre y rol del usuario
35     const usuarioBD = data.data;
36
37     //Guardamos en REDUX los datos del usuario autenticado
38     dispatch(authActions.login({
39       name: usuarioBD.nombre,
40       rol: usuarioBD.rol
41     }));
42
43     //Navegamos a la página principal
44     navigate("/home");
45
46   } catch (error) {
47     console.error("Error en el login:", error);
48     alert("Error en el servidor. Intentalo más tarde.");
49   }
50 }
```

El menú será común a todas las páginas de la aplicación: tanto si estamos en Inicio (en */home*) como si estamos en Informes (*/reports*) queremos ver el mismo menú. Por lo tanto lo vamos a programar en un fichero que estará dentro de la carpeta components: en el **frontend/src/components/Menu.tsx**.



App.tsx modificado:

```

Proyecto55 > backend > src > @AppInit -> src > components
12 import { useState } from "react";
13 import type { RootState } from "../store/index";
14 import Menu from "../components/Menu"; //IMPORTAMOS EL MENU
15 import React from "react";
16
17 //Componente para proteger las rutas
18 function ProtectedRoute({ children }: { children: React.ReactNode }) {
19   const isAuthenticated = useSelector(
20     (state: RootState) => state.authenticator.isAuthenticated
21   );
22   return isAuthenticated ? children : <Navigate to="/" replace />;
23 }
24
25 //Definimos las rutas del proyecto
26 const router = createBrowserRouter([
27   {
28     path: "/",
29     errorElement: <ErrorPage />, //Página de error personalizada
30     children: [
31       {
32         index: true, //Página principal
33         element: <Login />, //muestra Login al entrar en "/"
34       },
35     ],
36   },
37   {
38     path: "/home",
39     element: (
40       <ProtectedRoute>
41         { /* Envolvemos Home con el menú */ }
42         <Menu>
43         <Home />
44       </ProtectedRoute>
45     ),
46   },
47   {
48     path: "/reports",
49     element: (
50       <ProtectedRoute>
51         { /* Envolvemos Reports con el menú */ }
52         <Menu>
53         <Reports />
54       </ProtectedRoute>
55     ),
56   },
57 ],
58 );
59

```

4. Cómo evitar navegar a /home sin estar logueado: hook useEffect()

Hasta ahora si ponemos en la URL <http://localhost:5173/home> el enrutador nos montará el componente, que tendrá a su vez dentro el componente y el resto de componentes que tenga la página /home. Lo mismo pasa si ponemos a mano en el navegador la URL <http://localhost:5173/reports>, que el enrutador nos montará el componente que a su vez tiene dentro el componente y el resto de componente que tenga esa página. Es decir, entramos directamente en cualquiera de las dos páginas sin poner usuario ni contraseña.

Para evitar que pase eso se usa el hook useEffect().

(<https://react.dev/reference/react/useEffect>)

Como el componente común a ambas páginas (la de /home y la de /reports) es el <Menu/> que creamos en el punto 3, es ahí donde tendremos que colocar el hook useEffect().

Este hook permite sincronizar un componente cuando cambie algo externo. Ese algo externo se representa con las dependencias. Es decir, si cambian las dependencias, yo haré una acción en mi componente. Esas dependencias deben estar dentro de la lógica del useEffect(). En nuestro caso, si no estamos autenticados, queremos que se navegue a /. Por lo tanto las dependencias son estar o no autenticados y navegar.

Esta es la estructura de uso del hook useEffect:

```

useEffect(() => { //Lo
que queremos hacer
}, [dependencias])

```

Para saber si estamos o no autenticados, usamos los datos que obtuvimos del store con el useSelector(). Esos datos están almacenados en el objeto userData (o como lo hayan llamado ustedes en su código). Recordemos que ese objeto tiene los siguientes elementos, que nos dicen el nombre de usuario, el rol del usuario y si está o no autenticado:

```

userData.userName userData.userRol
userData.isAuthenticated

```

Con el hook useEffect() evitamos que se monte la página /home o la página /reports, puesto que reaccionará cuando se cambie el userData.isAuthenticated, comprobará si es false y si lo es, navegaremos a <http://localhost:5173/> (es decir, la ruta padre /).

Comprueba ahora que al poner `http://localhost:5173/home` o `http://localhost:5173/reports` directamente en la URL no te deja entrar puesto que no estás logueado.

Menu.tsx:

```
20 import { useState, useEffect } from "react";
```

```
37 const isAuthenticated = useSelector((state: RootState) => state.authenticator.userRol);
38
39 useEffect(() => {
40   if (!isAuthenticated) {
41     navigate('/')
42   }
43 }, [isAuthenticated, navigate])
```

5. Creación del componente que usarás en Home.tsx

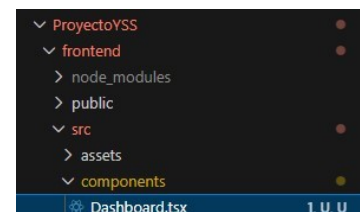
Crea un componente llamado Dashboard en el fichero `frontend/src/components/Dahsboard.tsx`. Este componente tendrá el formulario y la tabla que se usarán en la página `/home`.

```
import Dashboard from
'../components/
Dashboard'
Dashboard/
>
```

Es decir, ahora el fichero `Home.tsx` renderizará los componentes y en su return:

Dashboard.tsx:

```
ProyectoYSS > frontend > src > components > Dashboard.tsx > default
1 //Este componente contiene el FORMULARIO + TABLA que se mostrarán en /home
2 //Ambos elementos estarán en Home gracias a <Dashboard/>
3
4 import React from 'react';
5 import { Box, Typography, Paper } from '@mui/material';
6
7 const Dashboard: React.FC = () => {
8   return (
9     <Box sx={{ p: 3 }}>
10       <Paper elevation={3} sx={{ p: 3, mb: 3 }}>
11         <Typography variant="h5" component="h2" gutterBottom>
12           Formulario
13         </Typography>
14         <Typography variant="body1">
15           Formulario que se muestra en /home
16         </Typography>
17       </Paper>
18
19       <Paper elevation={3} sx={{ p: 3 }}>
20         <Typography variant="h5" component="h2" gutterBottom>
21           Tabla de Datos
22         </Typography>
23         <Typography variant="body1">
24           Tabla que muestra los datos en /home
25         </Typography>
26       </Paper>
27     </Box>
28   );
29 };
30
31 export default Dashboard;
```



En `home.tsx` lo único que hacemos es un import “

” y
dentro del return “

6. Creación del formulario en Dashboard.tsx

Ya hemos creado varios formularios a lo largo del curso, así que esto no debería ser complicado. Ten en cuenta que el formulario debe ser responsive.

Recuerda que todos los elementos del formulario: nombre, tipo, marca y precio los debes poner en un useState. Como estamos trabajando con Typescript deben indicar los tipos de los diferentes elementos, para ello hay que crear una interface:

El botón de + INSERTAR DATOS de nuestro formulario debe insertar datos en la tabla coleccion de nuestra base de datos.

Para ello debemos responder al evento onSubmit realizando una llamada al endpoint correspondiente.

(Nos vamos a la Actividad 6 de la UT1 donde creamos un formualrio para miAnimal y teniéndolo de ejemplo más unas modificaciones para adaptarlo a nuestro programa lo realizamos de la siguiente manera en Dashboard.tsx)

Dashboard.tsx:

```

1 import React, { useState } from 'react';
2
3 import {
4   Box,
5   Typography,
6   Paper,
7   Table,
8   Tablebody,
9   TableCell,
10  TableContainer,
11  Tablehead,
12  TableRow,
13  Chip,
14  Alert,
15  Stack,
16  TextField,
17  Button,
18  Grid
19 } from '@mui/material';
20
21 //Creamos el tipo ItemType, este tipo será un objeto con un id opcional de tipo number
22 //nombre, marca y tipo de tipo string y el precio de tipo number
23 interface ItemType {
24   id?: number;
25   nombre: string;
26   marca: string;
27   tipo: string;
28   precio: number;
29 }
30
31 //Inicializo los valores del item. Aquí no pongo el id porque no lo necesito
32 const itemInitialState: ItemType = {
33   nombre: '',
34   marca: '',
35   tipo: '',
36   precio: 0
37 }
38
39 const Dashboard: React.FC = () => {
40   //Estado para almacenar la lista de productos mostrados en la tabla
41   const [productos, setProductos] = useState<ItemType[]>([]);
42   //Estado para los datos del formulario actual
43   const [item, setItem] = useState<ItemType>(itemInitialState);
44   //Estado para controlar si el formulario está en proceso de envío
45   const [isSubmitting, setIsSubmitting] = useState(false);
46   //Estado para mensajes de éxito después de una inserción exitosa
47   const [success, setSuccess] = useState<string>('');
48   //Estado para mensajes de error durante el proceso de inserción
49   const [error, setError] = useState<string>('');
50
51   //Función que maneja los cambios en los campos del formulario
52   //Actualiza el estado del item con los nuevos valores ingresados
53   const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
54     const { name, value } = e.target;
55     setItem(prev => ({
56       ...prev,
57       //Convierte el precio a número, mantiene los demás campos como string
58       [name]: name === 'precio' ? parseFloat(value) || 0 : value
59     }));
60
61     //Limpia mensajes anteriores cuando el usuario comienza a escribir
62     if (error) setError('');
63     if (success) setSuccess('');
64   };
65
66   //Función principal que maneja el envío del formulario
67   //Realiza la inserción de datos en la base de datos mediante una petición POST
68   const handleSubmit = async (e: React.FormEvent) => {
69     e.preventDefault(); //Previene el comportamiento por defecto del formulario
70
71     //VALIDACIÓN DE CAMPOS OBLIGATORIOS
72
73     //Verifica que todos los campos requeridos
74     if (item.nombre.trim() || item.marca.trim() || item.tipo.trim()) {
75       setError('Los campos no pueden estar vacíos.');
```



```

93     const response = await fetch("http://localhost:3030/api/coleccion", {
94       method: "POST",
95       headers: {
96         "Content-Type": "application/json"
97       },
98       body: JSON.stringify(item) //Convierte el objeto a JSON
99     });
100
101     //Procesa la respuesta del servidor
102     const data = await response.json();
103
104     //Verifica si la inserción fue exitosa
105     if (!data.data || data.data.length === 0) {
106       throw new Error("Error al insertar el producto en la base de datos");
107     }
108
109     //ACTUALIZACIÓN DEL ESTADO LOCAL
110
111     //Agrega el nuevo producto a la lista mostrada en la tabla
112     setProductos(prev => [...prev, data.data]);
113
114     //FEEDBACK AL USUARIO
115
116     //Muestra mensaje de éxito y limpia el formulario
117     setSuccess("Producto insertado correctamente en la base de datos");
118     setItem(itemInitialState); //Restablece el formulario a valores iniciales
119
120   } catch (error) {
121     //Captura y muestra errores de conexión o del servidor
122     console.error("Error al insertar producto:", error);
123     setError(error instanceof Error ? error.message : "Error en el servidor. Intentalo más tarde.");
124   } finally {
125     //Siempre se ejecuta, haya éxito o error
126     setIsSubmitting(false); //Termina el estado de envío
127   }
128 };
129
130 return (
131   <Box sx={{ p: 3 }}>
132     /* SECCIÓN DEL FORMULARIO */
133     <Paper elevation={3} sx={{ p: 3, mb: 3, borderRadius: 2 }}>
134       <Typography variant="h5" component="h2" gutterBottom color="primary">
135         Registro de Productos
136       </Typography>
137       <Typography variant="body1" sx={{ mb: 2, color: 'text.secondary' }}>
138         Complete el formulario para agregar nuevos productos a la base de datos

```

```

141   /* ALERTAS DE FEEDBACK */
142   /* Muestra mensaje de éxito tras inserción exitosa */
143   <success && (
144     <Alert severity="success" sx={{ mb: 2 }}>
145       {success}
146     </Alert>
147   )
148
149   /* Muestra mensaje de error si ocurre algún problema */
150   <error && (
151     <Alert severity="error" sx={{ mb: 2 }}>
152       {error}
153     </Alert>
154   )
155
156   /* FORMULARIO PRINCIPAL */
157   <form onSubmit={handleSubmit}>
158     <Stack spacing={3}>
159       /* LAYOUT RESPONSIVE CON GRID */
160       <Grid container spacing={2}>
161
162       /* CAMPO NOMBRE */
163       <Grid size={{ xs: 12, sm: 6 }}>
164         <TextField
165           label="Nombre del Producto"
166           name="nombre"
167           value={item.nombre}
168           onChange={handleChange}
169           fullWidth
170           required
171           disabled={isSubmitting}
172           size="small"
173           placeholder="Ingresa el nombre del producto"
174         </>
175       </Grid>
176
177       /* CAMPO MARCA */
178       <Grid size={{ xs: 12, sm: 6 }}>
179         <TextField
180           label="Marca"
181           name="marca"
182           value={item.marca}
183           onChange={handleChange}
184           fullWidth
185           required
186           disabled={isSubmitting}

```



```

187         size="small"
188         placeholder="Ingrese la marca del producto"
189     />
190 </Grid>
191
192     { /* CAMPO TIPO */ }
193     <Grid size={{ xs: 12, sm: 6 }}>
194         <TextField
195             label="Tipo"
196             name="tipo"
197             value={item.tipo}
198             onChange={handleChange}
199             fullWidth
200             required
201             disabled={isSubmitting}
202             size="small"
203             placeholder="Ingrese el tipo de producto"
204         />
205     </Grid>
206
207     { /* CAMPO PRECIO */ }
208     <Grid size={{ xs: 12, sm: 6 }}>
209         <TextField
210             label="Precio"
211             name="precio"
212             type="number"
213             value={item.precio}
214             onChange={handleChange}
215             fullWidth
216             required
217             inputProps={{
218                 min: 0,
219                 step: 0.01
220             }}
221             disabled={isSubmitting}
222             size="small"
223             placeholder="0.00"
224         />
225     </Grid>
226 </Grid>
227
228     { /* BOTÓN DE ENVÍO */ }
229     <Button
230         variant="contained"
231         type="submit"
232         disabled={isSubmitting}

```

```

280         backgroundColor: '#ffff00', // Efecto hover suave
281         transition: 'background-color 0.2s'
282     }}
283 >
284 <Table>
285     <TableHeader>
286         <TableCell component="th" scope="row">
287             {producto.nombre}
288         </TableCell>
289         <TableCell>{producto.marca}</TableCell>
290         <TableCell>
291             { /* Chip para mostrar el tipo de producto */ }
292             <Chip
293                 label={producto.tipo}
294                 color="primary"
295                 variant="outlined"
296                 size="small"
297             />
298         </TableCell>
299         <TableCell align="right">
300             { /* Formato de precio con 2 decimales */ }
301             ${producto.precio.toFixed(2)}
302         </TableCell>
303         <TableCell>
304             { /* Indicador de estado del producto */ }
305             <Chip
306                 label="Activo"
307                 color="success"
308                 size="small"
309             />
310         </TableCell>
311     </TableHeader>
312     <TableBody>
313         { /* Contenido de la tabla */ }
314     </TableBody>
315 </Table>
316 </TableContainer>
317 </Box>
318 </div>
319 </div>
320 </div>
321 export default Dashboard;

```

```

233       sx={{
234         py: 1.2,
235         fontWeight: 'bold',
236         borderRadius: 2
237       }}
238       fullWidth
239     >
240     {/* Texto dinámico según estado del envío */}
241     {isSubmitting ? 'INSERTANDO EN BASE DE DATOS...' : '+ INSERTAR DATOS'}
242   </Button>
243 </Stack>
244 </form>
245 </Paper>
246
247 {/* SECCIÓN DE LA TABLA DE PRODUCTOS */}
248 <Paper elevation={3} sx={{ p: 3, borderRadius: 2 }}>
249   <Typography variant="h5" component="h2" gutterBottom color="secondary">
250     Inventario de Productos
251   </Typography>
252
253   {/* ESTADO DE LA TABLA */}
254   {/* Mensaje cuando no hay productos registrados */}
255   {productos.length === 0 ? (
256     <Typography variant="body1" color="text.secondary" textAlign="center" sx={{ py: 4 }}>
257       No hay productos registrados en la base de datos.
258     </Typography>
259   ) : (
260     /* TABLA CON LOS PRODUCTOS REGISTRADOS */
261     <TableContainer>
262       <Table sx={{ minWidth: 650 }} aria-label="tabla de productos">
263         <TableHead>
264           <TableRow>
265             <TableCell><strong>Nombre</strong></TableCell>
266             <TableCell><strong>Marca</strong></TableCell>
267             <TableCell><strong>Tipo</strong></TableCell>
268             <TableCell align="right"><strong>Precio</strong></TableCell>
269             <TableCell><strong>Estado</strong></TableCell>
270           </TableRow>
271         </TableHead>
272         <TableBody>
273           {/* ITERACIÓN SOBRE LOS PRODUCTOS */}
274           {productos.map((producto) => (
275             <TableRow
276               key={producto.id}
277               sx={{
278                 '&:last-child td, &:last-child th': { border: 0 },
279                 '&:hover': {
280                   backgroundColor: '#fff6fb', // Efecto hover suave
281                   transition: 'background-color 0.2s'
282                 }
283               }}
284             >
285               <TableCell component="th" scope="row">
286                 {producto.nombre}
287               </TableCell>
288               <TableCell>{producto.marca}</TableCell>
289               <TableCell>
290                 {/* Chip para mostrar el tipo de producto */}
291                 <Chip
292                   label={producto.tipo}
293                   color="primary"
294                   variant="outlined"
295                   size="small"
296                 />
297               </TableCell>
298               <TableCell align="right">
299                 {/* Formato de precio con 2 decimales */}
300                 ${producto.precio.toFixed(2)}
301               </TableCell>
302               <TableCell>
303                 {/* Indicador de estado del producto */}
304                 <Chip
305                   label="Activo"
306                   color="success"
307                   size="small"
308                 />
309               </TableCell>
310             </TableRow>
311           ))}
312         </TableBody>
313       </Table>
314     </TableContainer>
315   )}
316 </Paper>
317 </Box>
318 );
319 };
320
321 export default Dashboard;

```

7. Creación del fichero backend/services/items.js

Ahora vamos a saltar al backend de la aplicación para **crear las consultas que manejarán la comunicación con la tabla coleccion** de la base de datos bdgestion.sql. Contenido incompleto del fichero que irán rellenando:

```

services
JS coleccion.js
U, U

```

Coleccion.js:

```

frontend > src > services > # colección.js > @getdata
1 const db = require('../db')
2 const helper = require('../helper')
3 const config = require('../config')
4
5 //Función con la consulta para insertar datos en la base de datos: INSERT
6 async function insertData(req, res) {
7   //data tiene los datos que vamos a insertar en la base de datos. Los coge de req.query
8   //Para acceder a cada uno de los datos: data.nombre, data.precio, ...
9   const data = req.query
10
11   const result = await db.query(
12     `INSERT INTO coleccion (nombre, marca, tipo, precio)
13     VALUES (?, ?, ?, ?)`,
14     [data.nombre, data.marca, data.tipo, data.precio]
15   )
16
17   /*En la variable result se almacena lo que devuelve la consulta. Si accedemos a affectedRows nos da el número de filas de la base de
18   datos que ha sido modificado o añadido. Si ese número es mayor que cero es que ha habido inserción en la base de datos.*/
19
20   if (result.affectedRows) {
21     //Si la inserción fue exitosa, obtenemos el producto recién insertado
22     const [newProduct] = await db.query(
23       `SELECT id, nombre, marca, tipo, precio
24       FROM coleccion
25       WHERE id = ?`,
26       [result.insertId]
27     )
28     return {
29       message: 'Producto insertado correctamente',
30       data: newProduct[0]
31     }
32   }
33
34   return {
35     message: 'Error al insertar el producto',
36     data: []
37   }
38 }
39
40 //Función con la consulta de obtener datos de la base de datos: select * from coleccion
41 async function getData(req, res) {
42   //La variable rows almacena los datos obtenidos de la consulta select.
43   const rows = await db.query(
44     `SELECT id, nombre, marca, tipo, precio
45     FROM coleccion
46     ORDER BY id DESC`
47   )

```

```

49   /*Los datos obtenidos de la consulta del select los paso por la función helper para que
50   en el caso de que no haya datos devueltos, me devuelva un array vacío. */
51   const data = helper.emptyOrRows(rows)
52
53   return {
54     //Devolvemos el resultado del Select, que está almacenado en la variable data
55     data
56   }
57 }
58
59 //Función con la consulta para borrar datos de la base de datos: DELETE
60 async function deleteData(req, res) {
61   //En data almaceno los datos que me pasan para poder realizar el delete, me pasarán el id.
62   const data = req.query
63
64   const result = await db.query(
65     `DELETE FROM coleccion WHERE id = ?`,
66     [data.id]
67   )
68
69   /*En la variable result se almacena lo que devuelve la consulta. Si accedemos a affectedRows nos da el número de filas de la base de
70   datos que ha sido borrado. Si ese número es mayor que cero es que ha habido borrado en la base de datos.*/
71
72   if (result.affectedRows) {
73     return {
74       message: 'Producto eliminado correctamente',
75       affectedRows: result.affectedRows
76     }
77   }
78
79   return {
80     message: 'Error al eliminar el producto',
81     affectedRows: 0
82   }
83 }
84
85 //Al final del fichero exporto las funciones getData, insertData y deleteData
86 module.exports = {
87   getData,
88   insertData,
89   deleteData
90 }

```

Comprobación por terminal, personalmente desde la terminal de **VSCode** y usando **SQLite**:

```

PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> Get-Content "bdgestion.sql" | & "D:\Aplicaciones\SQLite\sqlite-tools-win-x64-3500400\sqlite3.exe" "bdgestion.db"
PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> & "D:\Aplicaciones\SQLite\sqlite-tools-win-x64-3500400\sqlite3.exe" "bdgestion.db"
SQLite version 3.50.4 2025-07-30 19:33:53
Enter ".help" for usage hints.
sqlite> .tables
coleccion  usuarios
sqlite> SELECT * FROM usuarios;
1|Patricia|patricia|123456789|admin
2|usuario|user|123456789|user

```

8. Añadir funcionalidad al botón Insertar del formulario.

Cuando se pique en el botón Insertar, hay que insertar los datos en la base de datos y avisar al usuario que se han insertado correctamente.

- Programar la consulta INSERT en la función *insertData* del fichero *backend/services/coleccion.js*

b) Añadir en el fichero backend/index.js un endpoint para añadir un registro en la base de datos. Yo al endpoint le puse el nombre /addItem pero lo pueden nombrar como quieran. Fíjate en el endpoint /login, es muy parecido el código. El código incompleto sería así:

```

6    //importamos coleccion
7    const coleccion = require('./services/coleccion')

44 //ENDPOINT PARA INSERTAR NUEVOS PRODUCTOS - /addItem
45 app.get('/addItem', async function(req, res, next) {
46     try {
47         //Llamamos a la función insertData que está en el fichero coleccion y le pasamos req
48         res.json(await coleccion.insertData(req, res))
49     } catch (err) {
50         console.error(`Error while inserting items `, err.message);
51         next(err);
52     }
53 })

54
55 //ENDPOINT PARA OBTENER PRODUCTOS
56 app.get('/api/coleccion', async function(req, res, next) {
57     try {
58         res.json(await coleccion.getData(req, res))
59     } catch (err) {
60         console.error(`Error while getting collection data`, err.message);
61         next(err);
62     }
63 })

```

c) En el frontend llamar al backend con la función fetch() cuando se pique el botón de Insertar. En la función que maneje el botón Insertar, debemos hacer un fetch al endpoint /addItem. Aquí fijarse en el fetch que hicieron en el login, es muy parecido. Si la respuesta del fetch es > 0, lanzamos un alert indicando que hemos insertado los datos correctamente (*alert('Datos guardados con éxito')*).

```

90     try {
91         //PETICIÓN HTTP POST al endpoint del backend
92         //Envía los datos del formulario en formato JSON
93         const response = await fetch("http://localhost:3030/addItem", {
94             method: "POST",
95             headers: {
96                 "Content-Type": "application/json"
97             },
98             body: JSON.stringify(item) //Convierte el objeto a JSON
99         });
100
101         //Procesa la respuesta del servidor
102         const data = await response.json();
103
104         //Verifica si la inserción fue exitosa
105         if (data.affectedRows && data.affectedRows > 0) {
106             alert('Datos guardados con éxito');
107
108             //Agrega el nuevo producto a la lista mostrada en la tabla
109             setProductos(prev => [...prev, { ...item, id: Date.now() }]);
110
111             //FEEDBACK AL USUARIO
112             //Muestra mensaje de éxito y limpia el formulario
113             setSuccess('Producto insertado correctamente en la base de datos');
114             setItem(itemInitialState); //Restablece el formulario a valores iniciales
115         } else {
116             throw new Error("Error al insertar el producto en la base de datos");
117         }
118     }

```

En index.js:

```

65 //endpoint /addItem para aceptar POST
66 app.post('/addItem', async function(req, res, next) {
67   try {
68     // Para POST usamos req.body en lugar de req.query
69     req.query = req.body; // Adaptamos para usar la misma función
70     res.json(await coleccion.insertData(req, res))
71   } catch (err) {
72     console.error(`Error while inserting items `, err.message);
73     next(err);
74   }
75 })

```

9. Creación del resto de los endpoints: /getItems y /deleteItem

9.1 Obtener datos de la base de datos (/getItems)

- Programar la consulta SELECT en la función getData del fichero backend/services/items.js
- Añadir en el fichero backend/index.js el endpoint para seleccionar los datos de la tabla colección. Yo al endpoint le puse el nombre /getItems pero lo pueden nombrar como quieran. El código incompleto sería así: *index.js*:

```

55 //ENDPOINT PARA OBTENER TODOS LOS PRODUCTOS - /getItems
56 app.get('/getItems', async function(req, res, next) {
57   try {
58     // Llamamos a la función getData que está en el fichero coleccion
59     res.json(await coleccion.getData(req, res))
60   } catch (err) {
61     console.error(`Error while getting items `, err.message);
62     next(err);
63   }
64 })

```

Backend:



Backend\services\coleccion.js:


```

ProyectoVSS > Backend > services > db.js > @ default
1 import db from './db.js';
2 import helper from './helper.js';
3
4 //Función para insertar datos - CORREGIDA para SQLite
5 async function insertData(req, res) {
6   const data = req.query;
7
8   try {
9     const result = await db.run(
10       `INSERT INTO coleccion (nombre, marca, tipo, precio)
11       VALUES (?, ?, ?, ?)`,
12       [data.nombre, data.marca, data.tipo, data.precio]
13     );
14
15     // CORRECCIÓN: SQLite devuelve el resultado diferente
16     // devuelve solo el número de filas afectadas como tu amigo
17     let affectedRows = 0;
18     if (result && result.affectedRows) {
19       affectedRows = result.affectedRows;
20     }
21
22     console.log('Insert result:', result, 'Affected rows:', affectedRows);
23     return affectedRows;
24   } catch (error) {
25     console.error('Error en insertData:', error);
26     return 0;
27   }
28 }
29
30 //Función para obtener datos - CORREGIDA para SQLite
31 async function getData(req, res) {
32   try {
33     const rows = await db.query(
34       `SELECT id, nombre, marca, tipo, precio
35       FROM coleccion
36       ORDER BY id DESC
37       `);
38
39     console.log('Data from SQLite:', rows);
40
41     const data = helper.emptyObject(rows);
42
43     return {
44       data
45     };
46   } catch (error) {
47     console.error('Error en getData:', error);
48     return {
49       data: []
50     };
51   }
52 }

```

```

53 }
54
55 //Función para borrar datos - CORREGIDA para SQLite
56 async function deleteData(req, res) {
57   const data = req.query;
58
59   try {
60     const result = await db.run(
61       `DELETE FROM coleccion WHERE id = ?`,
62       [data.id]
63     );
64
65     //SQLite devuelve el resultado diferente
66     let affectedRows = 0;
67     if (result && result.affectedRows) {
68       affectedRows = result.affectedRows;
69     }
70
71     console.log('Delete result:', result, 'Affected rows:', affectedRows);
72     return affectedRows;
73   } catch (error) {
74     console.error('Error en deleteData:', error);
75     return 0;
76   }
77 }
78
79 export default {
80   getData,
81   insertData,
82   deleteData
83 };

```

Backend/services/db.js

```

ProyectoVSS > Backend > services > db.js > @ default
1 import sqlite3 from 'sqlite3';
2 import path from 'path';
3 import { fileURLToPath } from 'url';
4
5 const __filename = fileURLToPath(import.meta.url);
6 const __dirname = path.dirname(__filename);
7
8 const dbPath = path.join(__dirname, '..', 'bdgestion.db');
9
10 // VERIFICA que la base de datos existe
11 console.log('Database path:', dbPath);
12
13 async function query(sql, params = []) {
14   return new Promise((resolve, reject) => {
15     const db = new sqlite3.Database(dbPath);
16
17     db.all(sql, params, (err, rows) => {
18       if (err) {
19         console.error('SQLite query error:', err);
20         reject(err);
21       } else {
22         console.log('SQLite query result:', rows);
23         resolve(rows);
24       }
25     });
26     db.close();
27   });
28 }
29
30 async function run(sql, params = []) {
31   return new Promise((resolve, reject) => {
32     const db = new sqlite3.Database(dbPath);
33
34     db.run(sql, params, function(err) {
35       if (err) {
36         console.error('SQLite run error:', err);
37         reject(err);
38       } else {
39         console.log('SQLite run result:', this);
40         resolve({
41           affectedRows: this.changes,
42           insertId: this.lastID
43         });
44       }
45     });
46     db.close();
47   });
48 }
49
50 export default {
51   query,
52   run
53 }

```

Backend/services/login.js:

```

ProyectoVSS > Backend > services > login.js > @ default
1 import db from './db.js';
2
3 async function getUserData(user, password) {
4   try {
5     const rows = await db.query(
6       `SELECT nombre, rol
7       FROM usuarios
8       WHERE login = ? AND password = ?`,
9       [user, password]
10    );
11
12    //Comprobación directa sin helper
13    const data = rows && rows.length > 0 ? rows[0] : null;
14
15    return {
16      data
17    };
18   } catch (error) {
19     console.error('Error en getUserData:', error);
20     return {
21       data: null
22     };
23   }
24 }
25
26 //Exportación correcta para ES modules
27 export default {
28   getUserData
29 };

```

Backend/config.js:

```

ProyectoYSS > Backend > JS config.js > default
1  const config = {
2      db: {
3          host: 'localhost',
4          user: 'tu_usuario',
5          password: 'tu_password',
6          database: 'bdgestion',
7          port: 3306
8      }
9  };
10
11  // Cambiar a export default
12  export default config;

```

Backend/index.js:

```

//ProyectoYSS > Backend > JS index.js > ...
1  //importar usando ES modules
2  import express from 'express';
3  import cors from 'cors';
4  import login from './services/login.js';
5  import coleccion from './services/coleccion.js';
6
7  const port = 3030;
8  const app = express();
9
10 app.use(express.json());
11 app.use(express.urlencoded({ extended: true }));
12 app.use(cors());
13
14 app.get('/', function (req, res) {
15   | res.json({ message: 'Hola Mundo!' });
16 });
17
18 app.get('/login', async function(req, res, next) {
19   | console.log(req.query);
20   | console.log(req.query.user);
21   | console.log(req.query.password);
22   | try {
23   |   | res.json(await login.getterData(req.query.user, req.query.password));
24   | } catch (err) {
25   |   | console.error('Error while getting data ', err.message);
26   |   | next(err);
27   | }
28 });
29
30 //ENDPOINT PARA INSERTAR NUEVOS PRODUCTOS - /addItem
31 app.get('/addItem', async function(req, res, next) {
32   | try {
33   |   | //Llamamos a la función insertData que está en el fichero coleccion y le pasamos req
34   |   | res.json(await coleccion.insertData(req, res))
35   | } catch (err) {
36   |   | console.error('Error while inserting items ', err.message);
37   |   | next(err);
38   | }
39 });
40
41 //ENDPOINT PARA OBTENER TODOS LOS PRODUCTOS - /getItems
42 app.get('/getItems', async function(req, res, next) {
43   | try {
44   |   | //Llamamos a la función getData que está en el fichero coleccion
45   |   | res.json(await coleccion.getData(req, res))
46   | } catch (err) {
47   |   | console.error('Error while getting items ', err.message);
48   |   | next(err);
49   | }
50 });

```

```

52 //endpoint /addItem para aceptar POST
53 app.post('/addItem', async function(req, res, next) {
54   | try {
55   |   | // Para POST usamos req.body en lugar de req.query
56   |   | req.query = req.body; // Adaptamos para usar la misma función
57   |   | res.json(await coleccion.insertData(req, res))
58   | } catch (err) {
59   |   | console.error('Error while inserting items ', err.message);
60   |   | next(err);
61   | }
62 });
63
64 //ENDPOINT PARA ELIMINAR PRODUCTOS
65 app.get('/deleteItem', async function(req, res, next) {
66   | try {
67   |   | res.json(await coleccion.deleteData(req, res))
68   | } catch (err) {
69   |   | console.error('Error while deleting items ', err.message);
70   |   | next(err);
71   | }
72 });
73
74 //Iniciamos la API
75 app.listen(port)
76 console.log('API escuchando en el puerto ' + port)

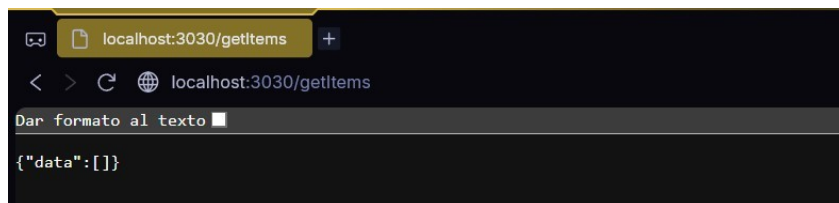
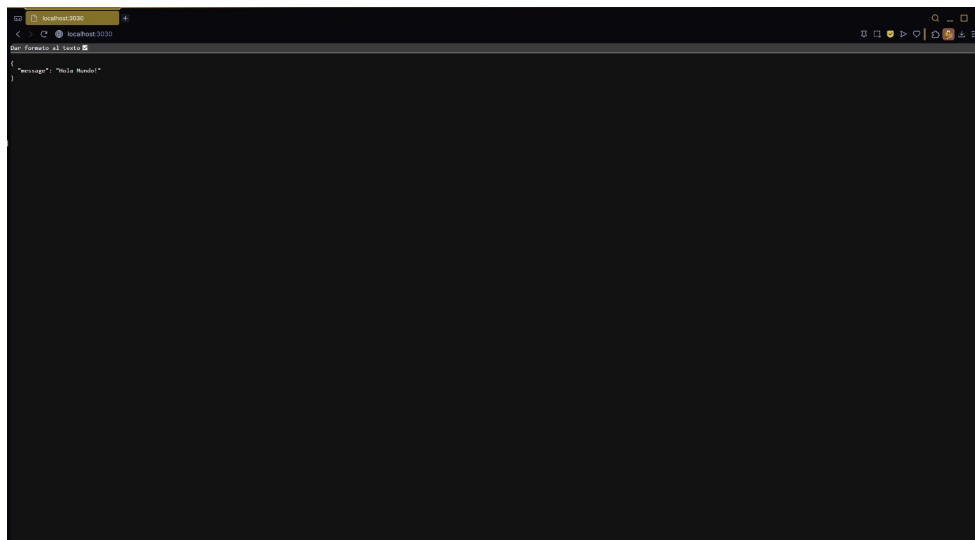
```

Para probar que funciona vamos al endpoint /getItems a través del navegador (recuerden que los endpoints están en el puerto 3030): <http://localhost:3030/getItems> Veremos cómo son los datos que nos devuelve la base de datos. Si en la tabla coleccion la base de datos aún está vacía nos devuelve un objeto vacío:

```

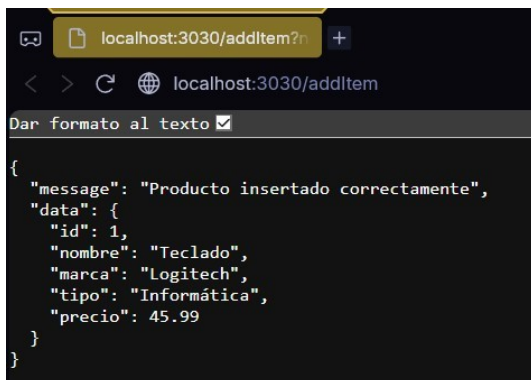
PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS\Backend> node index.js
API escuchando en el puerto 3030

```

Para añadir un registro a la tabla ponemos, por ejemplo, lo siguiente en el navegador:

<http://localhost:3030/addItem?nombre=Teclado&marca=Logitech&tipo=Informática&precio=45.99>



9.2 Borrar un registro de la base de datos (/deleteItem)

a) Programar la consulta DELETE en la función deleteData del fichero backend/services/items.js b) Añadir en el fichero *backend/index.js* el endpoint para seleccionar los datos de la tabla coleccion. Yo al endpoint le puse el nombre /deleteItem pero lo pueden nombrar como quieran. El código incompleto sería así:

```
64 //ENDPOINT PARA ELIMINAR PRODUCTOS
65 app.delete('/deleteItem', async function(req, res, next) {
66     try {
67         res.json(await coleccion.deleteData(req, res))
68     } catch (err) {
69         console.error(`Error while deleting items `, err.message);
70         next(err);
71     }
72 }
```

Para probar que nos funciona el borrar items, vamos a insertar varios, como en el apartado anterior:

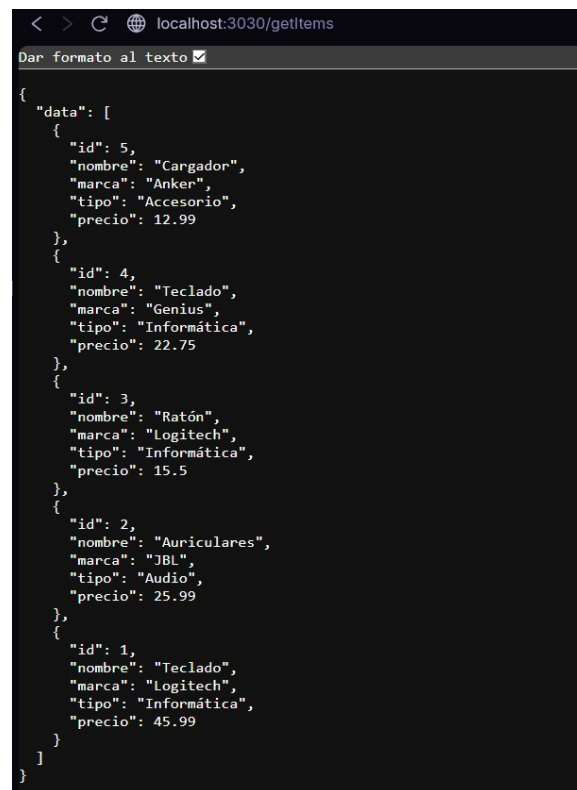
<http://localhost:3030/addItem?nombre=Auriculares&marca=JBL&tipo=Audio&precio=25.99>

<http://localhost:3030/addItem?nombre=Ratón&marca=Logitech&tipo=Informática&precio=15.50>

<http://localhost:3030/addItem?nombre=Teclado&marca=Genius&tipo=Informática&precio=22.75>

<http://localhost:3030/addItem?nombre=Cargador&marca=Anker&tipo=Accesorio&precio=12.99>

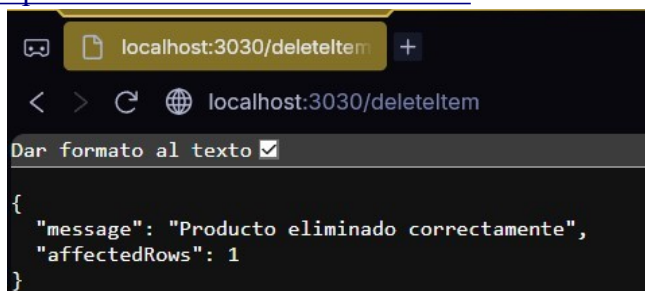
Esto se ve así:



```
{
  "data": [
    {
      "id": 5,
      "nombre": "Cargador",
      "marca": "Anker",
      "tipo": "Accesorio",
      "precio": 12.99
    },
    {
      "id": 4,
      "nombre": "Teclado",
      "marca": "Genius",
      "tipo": "Informática",
      "precio": 22.75
    },
    {
      "id": 3,
      "nombre": "Ratón",
      "marca": "Logitech",
      "tipo": "Informática",
      "precio": 15.5
    },
    {
      "id": 2,
      "nombre": "Auriculares",
      "marca": "JBL",
      "tipo": "Audio",
      "precio": 25.99
    },
    {
      "id": 1,
      "nombre": "Teclado",
      "marca": "Logitech",
      "tipo": "Informática",
      "precio": 45.99
    }
  ]
}
```

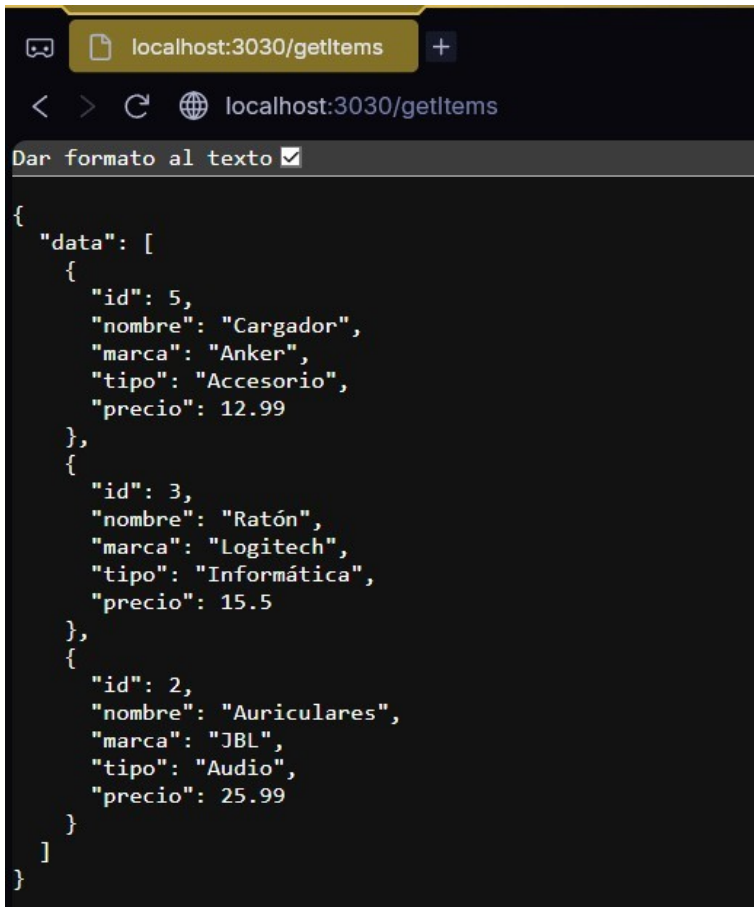
Vamos a borrar los productos con el id 1:

<http://localhost:3030/deleteItem?id=1>



```
{
  "message": "Producto eliminado correctamente",
  "affectedRows": 1
}
```

Comprobamos (también borré el ID 4 pero no le tomé captura sin querer):



```
{
  "data": [
    {
      "id": 5,
      "nombre": "Cargador",
      "marca": "Anker",
      "tipo": "Accesorio",
      "precio": 12.99
    },
    {
      "id": 3,
      "nombre": "Ratón",
      "marca": "Logitech",
      "tipo": "Informática",
      "precio": 15.5
    },
    {
      "id": 2,
      "nombre": "Auriculares",
      "marca": "JBL",
      "tipo": "Audio",
      "precio": 25.99
    }
  ]
}
```

10. Creación de la tabla en el fichero Dashboard.tsx: lógica para mostrar los datos en la tabla y lógica para borrar un registro de la tabla

10.1 Lógica para mostrar los datos en la tabla

Con la tabla mostramos los datos que están almacenados en la tabla colección de nuestra base de datos. Para mostrar los datos tenemos que hacer un SELECT de la tabla colección, almacenar dichos datos en una variable (que será un array) y luego mostrar los datos de la variable en la tabla.

Así que lo primero que tendremos que hacer es crear una variable en el fichero *frontend/src/components/Dashboard.tsx* en la que almacenaremos los datos obtenidos del SELECT. Como esa variable va a actualizarse cada vez que hagamos el select, la ponemos como un `useState`. Su valor inicial será un array vacío. Cuando hagamos el select, su valor se actualizará con los valores obtenidos de la base de datos.

Una vez tenemos claro en qué parte hacemos el `fetch()` al endpoint que realiza el SELECT y tenemos ya almacenados los datos de la tabla colección en la variable `tableData` ... vamos a crear la tabla.

```
148 | //Función para eliminar un producto
149 | const handleEliminarProducto = async (id: number) => {
150 |   if (window.confirm('¿Estás seguro de que quieres eliminar este producto?')) {
151 |     try {
152 |       const response = await fetch(`http://localhost:3030/deleteItem?id=${id}`, {
153 |         method: 'DELETE'
154 |       });
155 |
156 |       const data = await response.json();
157 |
158 |       if (data.affectedRows && data.affectedRows > 0) {
159 |         //Mostrar mensaje de éxito
160 |         setSuccess('Producto eliminado correctamente');
161 |
162 |         //Recargar la lista de productos
163 |         await cargarProductos();
164 |
165 |         //Limpiar mensaje después de 3 segundos
166 |         setTimeout(() => setSuccess(''), 3000);
167 |       } else {
168 |         throw new Error("Error al eliminar el producto");
169 |       }
170 |     } catch (error) {
171 |       console.error("Error al eliminar producto:", error);
172 |       setError('Error al eliminar el producto de la base de datos');
173 |
174 |       //Limpiar mensaje de error después de 3 segundos
175 |       setTimeout(() => setError(''), 3000);
176 |     }
177 |   }
178 | };
```

```

297 | | /* SECCIÓN DE LA TABLA DE PRODUCTOS */
298 | <Paper elevation={3} sx={{ p: 3, borderRadius: 2 }}>
299 |   <Typography variant="h5" component="h2" gutterBottom color="secondary">
300 |     Inventario de Productos
301 |   </Typography>
302 |
303 |   /* ALERTAS DE FEEDBACK PARA ELIMINACIÓN */
304 |   {success && (
305 |     <Alert severity="success" sx={{ mb: 2 }}>
306 |       {success}
307 |     </Alert>
308 |   )}
309 |
310 |   {error && (
311 |     <Alert severity="error" sx={{ mb: 2 }}>
312 |       {error}
313 |     </Alert>
314 |   )}
315 |
316 |   /* ESTADO DE LA TABLA */
317 |   {productos.length === 0 ? (
318 |     <Typography variant="body1" color="text.secondary" textAlign="center" sx={{ py: 4 }}>
319 |       No hay productos registrados en la base de datos.
320 |     </Typography>
321 |   ) : (
322 |     /* TABLA CON LOS PRODUCTOS REGISTRADOS */
323 |     <TableContainer>
324 |       <Table sx={{ minWidth: 650 }} aria-label="tabla de productos">
325 |         <TableHead>
326 |           <TableRow>
327 |             <TableCell><strong>ID</strong></TableCell>
328 |             <TableCell><strong>Nombre</strong></TableCell>
329 |             <TableCell><strong>Marca</strong></TableCell>
330 |             <TableCell><strong>Tipo</strong></TableCell>
331 |             <TableCell align="right"><strong>Precio</strong></TableCell>
332 |             <TableCell><strong>Acciones</strong></TableCell>
333 |           </TableRow>
334 |         </TableHead>
335 |         <TableBody>
336 |           /* ITERACIÓN SOBRE LOS PRODUCTOS */
337 |           {productos.map((producto) => (
338 |             <TableRow
339 |               key={producto.id}
340 |               sx={{
341 |                 '&:last-child td, &:last-child th': { border: 0 },
342 |                 '&:hover': {
343 |                   backgroundColor: '#fff6fb',
344 |                   transition: 'background-color 0.2s'
345 |                 }
346 |               }}

```

```

347 |             >
348 |               <TableCell>{producto.id}</TableCell>
349 |               <TableCell component="th" scope="row">
350 |                 {producto.nombre}
351 |               </TableCell>
352 |               <TableCell>{producto.marca}</TableCell>
353 |               <TableCell>
354 |                 <Chip
355 |                   label={producto.tipo}
356 |                   color="primary"
357 |                   variant="outlined"
358 |                   size="small"
359 |                 />
360 |               </TableCell>
361 |               <TableCell align="right">
362 |                 ${producto.precio.toFixed(2)}
363 |               </TableCell>
364 |               <TableCell>
365 |                 <Button
366 |                   variant="outlined"
367 |                   color="error"
368 |                   size="small"
369 |                   onClick={() => handleEliminarProducto(producto.id!)}
370 |                   startIcon={<🗑️/>}
371 |                   sx={{
372 |                     fontWeight: 'bold',
373 |                     '&:hover': {
374 |                       backgroundColor: '#ffebee'
375 |                     }
376 |                   }}
377 |                 >
378 |                   Eliminar
379 |                 </Button>
380 |               </TableCell>
381 |             </TableRow>
382 |           )}
383 |         </TableBody>
384 |       </Table>
385 |     </TableContainer>
386 |   )}
387 | </Paper>
388 | </Box>
389 | );
390 | };

```

10.2 Implementación de la tabla: componente Table

Para la tabla usamos el componente Table de la librería @MUI:

<https://mui.com/materialui/reacttable/>

Una tabla tiene la siguiente estructura básica:

- Contenedor de tabla (TableContainer)
- La tabla en sí (Table)

→ La cabecera de la tabla (TableHead) con un componente fila (TableRow) y luego cada una de las celdas (TableCell) de la fila de la cabecera.

→ El cuerpo de la tabla (TableBody) que tendrá varios componentes fila con varias celdas cada uno de ellos.

En nuestro caso vamos a rellenar el cuerpo de la tabla con los datos de la base de datos, que vienen en un array de objetos. Para recorrer un array en javascript se usa el método map como ya saben. Así pues, vemos a continuación el esquema de una tabla y la parte donde se haría el .map del array:

Fijarse que tenemos una propiedad llamada aria-label, que nos sirve para temas de accesibilidad, eso facilita la lectura de la aplicación a una persona con problemas de visión. Hay que ponerlo en el código.

Lo que queremos hacer ahora es mostrar con el componente esos datos obtenidos del endpoint /getItems y almacenados en el array tableData. Nos centramos ahora en lo que se pondría dentro del Body de la tabla, que es un .map con el que recorreremos el array tableData (siempre y cuando haya datos, si no los hay no se entra en el map). La variable row toma el valor de los elementos del array a medida que se va recorriendo. Así que row.id será el valor del id, row.nombre será el valor del nombre, ... Fíjense que row es de tipo itemtype, que fue el que definimos al principio del código de Home.tsx. Fíjense también que para el key usamos el id que proviene de la base de datos.

```
20 import DeleteForeverIcon from '@mui/icons-material/DeleteForever';
```

```
324 /* TABLA CON LOS PRODUCTOS REGISTRADOS */
325 <TableContainer>
326 <Table sx={{ minWidth: 650 }} aria-label="tabla de productos">
327   <TableHead>
328     <TableRow>
329       <TableCell><strong>Acciones</strong></TableCell>
330       <TableCell><strong>ID</strong></TableCell>
331       <TableCell><strong>Nombre</strong></TableCell>
332       <TableCell><strong>Marca</strong></TableCell>
333       <TableCell><strong>Tipo</strong></TableCell>
334       <TableCell align="right"><strong>Precio</strong></TableCell>
335     </TableRow>
336   </TableHead>
337   <TableBody>
338     {productos.map((row: itemtype) => (
339       <TableRow key={row.id}>
340         <TableCell>
341           <Button
342             onClick={() => handleEliminarProducto(row.id!)}
343             color="error"
344             size="small"
345             startIcon={<DeleteForeverIcon />}
346           >
347             Eliminar
348           </Button>
349         </TableCell>
350         <TableCell>{row.id}</TableCell>
351         <TableCell component="th" scope="row">{row.nombre}</TableCell>
352         <TableCell>{row.marca}</TableCell>
353         <TableCell>
354           <Chip label={row.tipo} color="primary" variant="outlined" size="small" />
355         </TableCell>
356         <TableCell align="right">${(row.precio.toFixed(2))}</TableCell>
357       </TableRow>
358     ))}
359   </TableBody>
360 </Table>
361 </TableContainer>
```

10.3 Lógica para borrar un registro de la tabla

En el código del TableBody hemos insertado un componente con un componente icono (). Al picar en dicho botón icono, tendremos que implementar una función a la que pasamos el id del registro que queremos borrar. Dicha función hará el fetch al endpoint /deleteItem. Ya está el propio código mostrado en el apartado anterior.

11. Ejecución del programa:

Primero tenemos que asegurarnos de que estamos corriendo por una terminal el Backend:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> cd backend
○ PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS\backend> node index.js
Database path: F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS\backend\bdgestion.db
API escuchando en el puerto 3030
```

Y por otra ejecutamos el frontend mientras:

```
● PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> cd frontend
○ PS F:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS\frontend> npm run dev

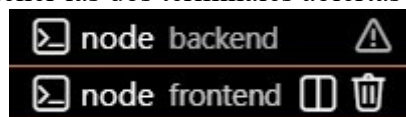
> frontend@0.0.0 dev
> vite

[vite:react-swc] We recommend switching to '@vitejs/plugin-react' for improved performance as no swc plugins are used. More information at https://vite.dev/rolldown

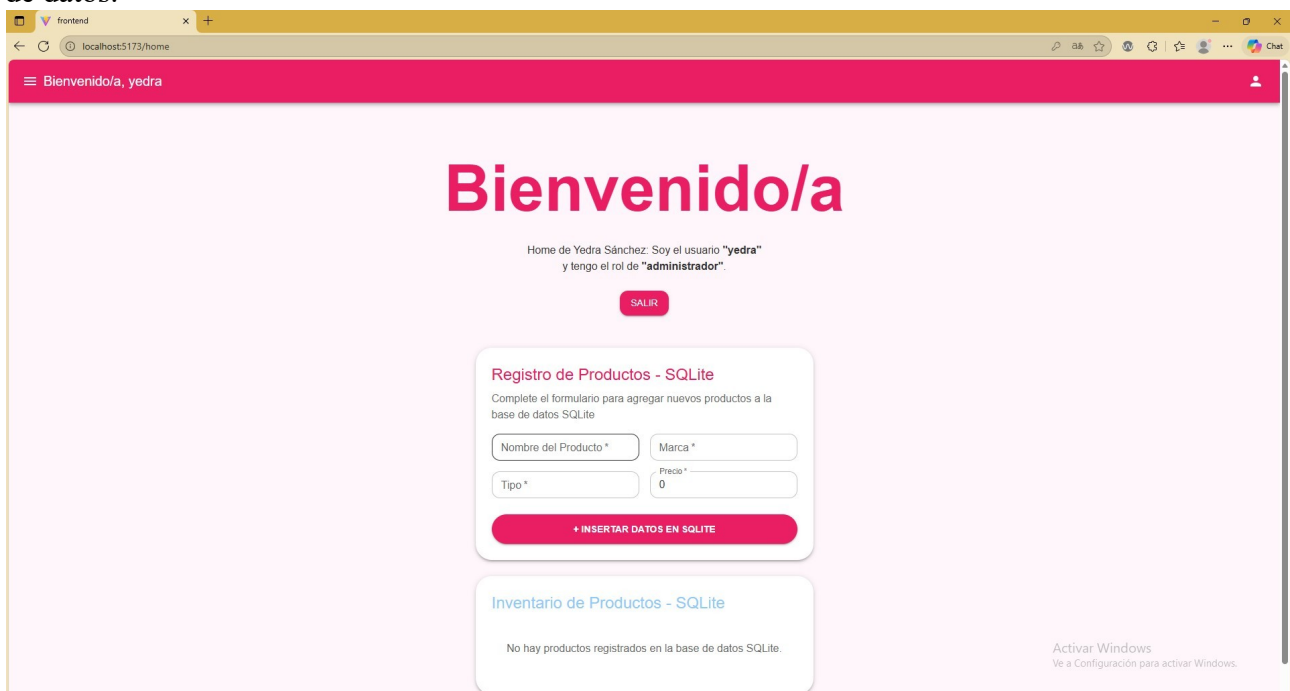
ROLLDOWN-VITE v7.1.14 ready in 292 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

En el propio VSC te deja tener las dos terminales abiertas y funcionando a la vez:



Una vez iniciamos sesión, entramos en la página principal donde podemos gestionar la base de datos.



Insertamos un producto:

Bienvenido/a, yedra

Bienvenido/a

Home de Yedra Sánchez: Soy el usuario "yedra" y tengo el rol de "administrador".

SALIR

Registro de Productos - SQLite

Complete el formulario para agregar nuevos productos a la base de datos SQLite

Nombre del Producto *
croisant

Marca *
Editesa

Tipo *
Alimento

Precio *
12

+ INSERTAR DATOS EN SQLITE

Comprobamos:

Home de Yedra Sánchez: Soy el usuario "yedra" y tengo el rol de "administrador".

SALIR

Registro de Productos - SQLite

Complete el formulario para agregar nuevos productos a la base de datos SQLite

Nombre del Producto *

Marca *

Tipo *

Precio *

0

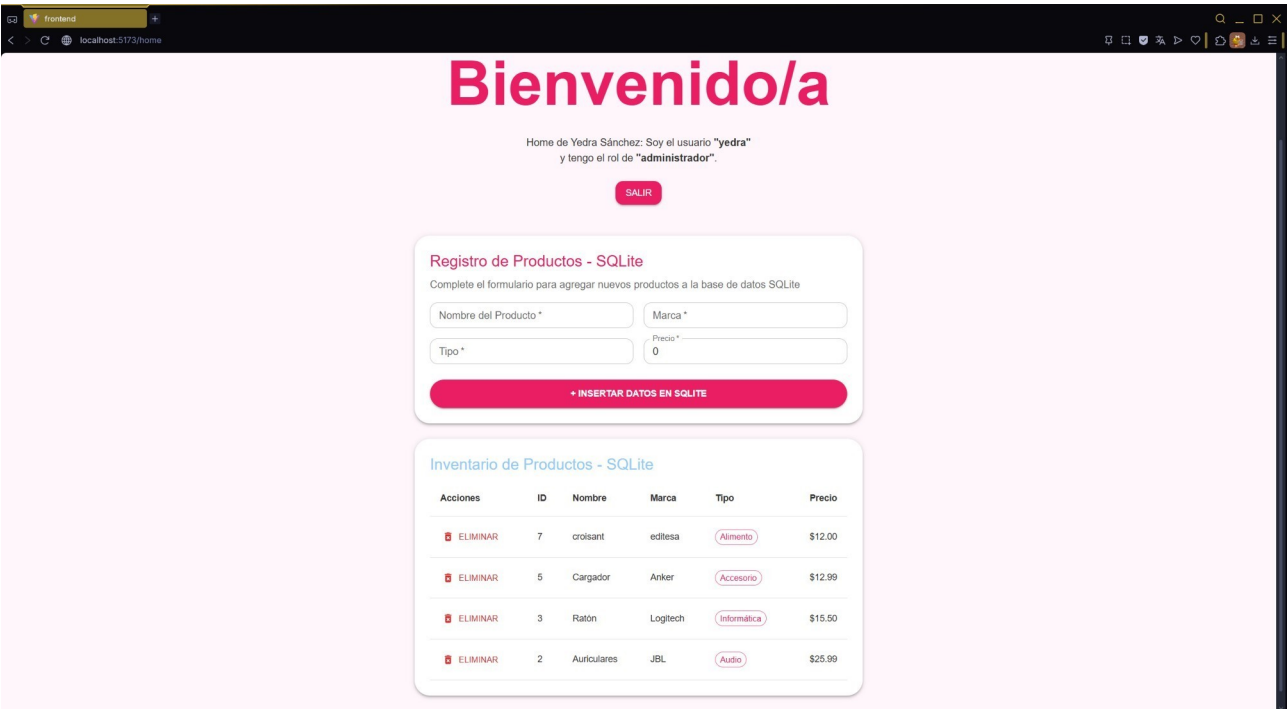
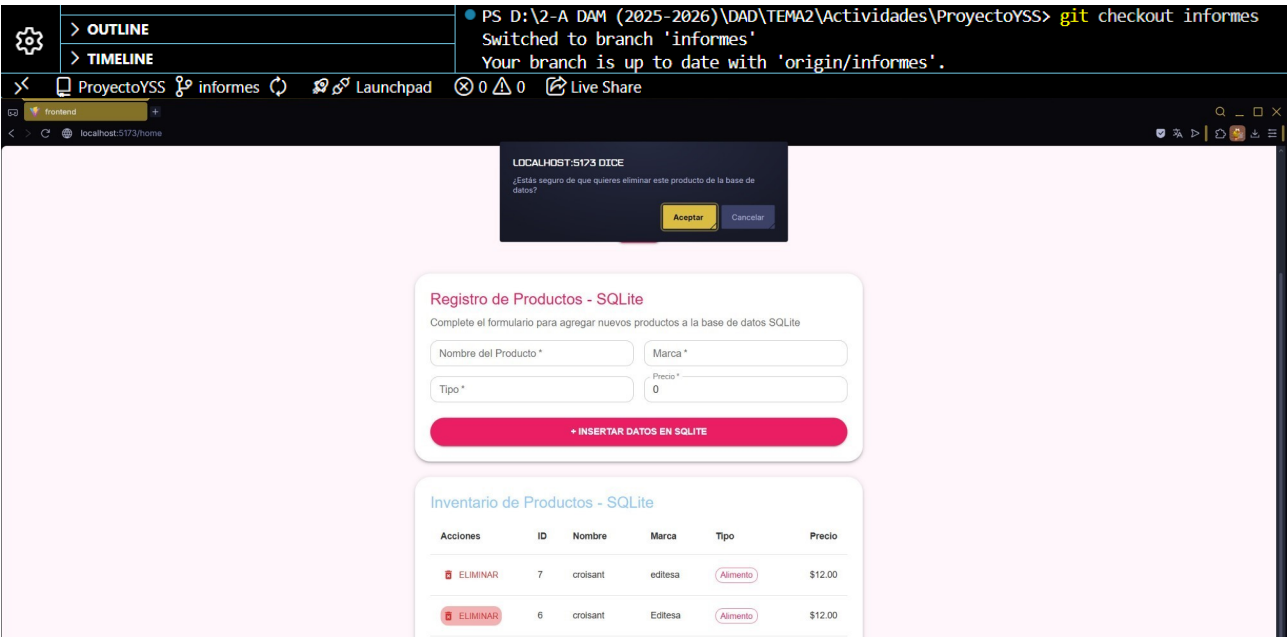
+ INSERTAR DATOS EN SQLITE

Inventario de Productos - SQLite

Acciones	ID	Nombre	Marca	Tipo	Precio
ELIMINAR	7	croisant	editesa	Alimento	\$12.00
ELIMINAR	6	croisant	Editesa	Alimento	\$12.00
ELIMINAR	5	Cargador	Anker	Accesorio	\$12.99
ELIMINAR	3	Ratón	Logitech	Informática	\$15.50
ELIMINAR	2	Auriculares	JBL	Audio	\$25.99

(Se duplicó porque soy una impaciente y lo puse 2 veces)

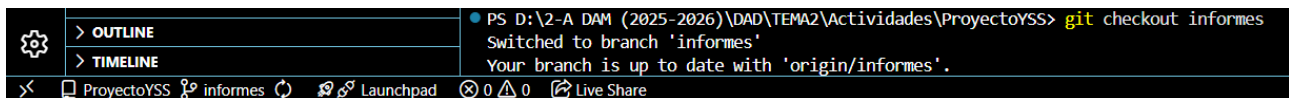
Ahora eliminamos desde el propio frontend con el icono ELIMINAR:



Actividad evaluable UT4A1 – Documentación de aplicaciones

1. Rama del proyecto

Nos movemos a la rama 'informes' y nos aseguramos de estar en ella:



```

PS D:\2-A DAM (2025-2026)\DAD\TEMA2\Actividades\ProyectoYSS> git checkout informes
Switched to branch 'informes'
Your branch is up to date with 'origin/informes'.
  
```

2. Creación de ayuda contextual: componente Tooltip

Usando el componente *Tooltip* de la librería *@MUI* crea ayuda contextual en todos los botones de la aplicación, tanto los que tienen icono como los que no.

Tal como indica la *documentación oficial de MUI* (<https://mui.com/material-ui/react-tooltip/>), para usar el componente *Tooltip* con forma de flecha y decidiendo su posición, debemos envolver los botones y utilizar las propiedades *arrow* y *placement*.

Nota: En los botones que utilizan la propiedad *disabled* (como los de enviar formularios mientras cargan), el componente *Tooltip* de *MUI* no detecta el evento *hover*. Para solucionarlo, esos botones se han envuelto en una etiqueta **.

Primero hay que importar *Tooltip* de *@mui/material* en cada uno de los archivos y modificar los botones.

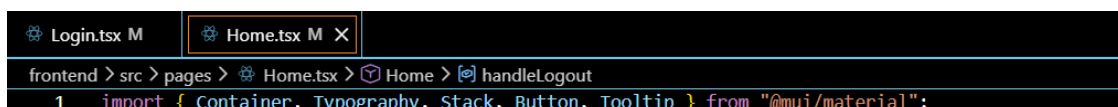
- frontend/src/pages/Login.tsx:



```

Login.tsx M X
frontend > src > pages > Login.tsx > Login
4 import React, { useState } from "react";
5
6 import {
7   Container,
8   TextField,
9   Button,
10  Typography,
11  Card,
12  CardContent,
13  Stack,
14  Box,
15  Alert,
16  Tooltip, // componente para mostrar información al pasar el mouse sobre el botón
17 } from "@mui/material"; //Imports escritos así para mejor entendimiento y repartición
18
139
140   <Tooltip title="Iniciar sesión en el sistema" placement="bottom" arrow>
141     <Button variant="contained" fullWidth type="submit">
142       ACCEDER
143     </Button>
144   </Tooltip>
  
```

- frontend/src/pages/Home.tsx:



```

Login.tsx M Home.tsx M X
frontend > src > pages > Home.tsx > Home > handleLogout
1 import { Container, Typography, Stack, Button, Tooltip } from "@mui/material";
  
```

```

48 | <Tooltip title="Cerrar sesión de forma segura" placement="right" arrow>
49 |   <Button
50 |     variant="contained"
51 |     color="primary"
52 |     onClick={handleLogout}
53 |   >
54 |     Salir
55 |   </Button>
56 | </Tooltip>

```

- frontend/src/components/Dashboard.tsx:

```

Login.tsx M Home.tsx M Dashboard.tsx M X
frontend > src > components > Dashboard.tsx > ...
1  import React, { useState, useEffect } from 'react';
2  import {
3    Box,
4    Typography,
5    Paper,
6    Table,
7    TableBody,
8    TableCell,
9    TableContainer,
10   TableHead,
11   TableRow,
12   Chip,
13   Alert,
14   Stack,
15   TextField,
16   Button,
17   Grid,
18   Tooltip
19 } from '@mui/material';

```

```

287 |   {/* BOTÓN DE ENVÍO */}
288 |   <Tooltip title="Añadir nuevo producto a SQLite" placement="top" arrow>
289 |     <span>
290 |       <Button
291 |         variant="contained"
292 |         type="submit"
293 |         disabled={isSubmitting}
294 |         sx={{ py: 1.2, fontWeight: 'bold', borderRadius: 2 }}
295 |         fullWidth
296 |       >
297 |         {isSubmitting ? 'INSERTANDO EN SQLite...' : '+ INSERTAR DATOS EN SQLite'}
298 |       </Button>
299 |     </span>
300 |   </Tooltip>
301 | </Stack>
302 | </form>
303 | </Paper>

```

```

351 |   {userRol === 'admin' && (
352 |     <Tooltip title="Eliminar este producto permanentemente" placement="left" arrow>
353 |       <Button
354 |         onClick={() => handleEliminarProducto(row.id!)}
355 |         color="error"
356 |         size="small"
357 |         startIcon={<DeleteForeverIcon />}
358 |         sx={{ '&:hover': { backgroundColor: '#ffebee' } }}
359 |       >
360 |         Eliminar
361 |       </Button>
362 |     </Tooltip>
363 |   )}

```

- frontend/src/components/InformeColeccion.tsx:

```

179 | <Box>
180 |   <Tooltip title="Descargar datos en formato CSV" placement="bottom" arrow>
181 |     <IconButton onClick={handleExportCSV} sx={{ color: 'white' }}><DownloadIcon /></IconButton>
182 |   </Tooltip>
183 |   <Tooltip title="Descargar informe en PDF" placement="bottom" arrow>
184 |     <IconButton onClick={handleExportPDF} sx={{ color: 'white' }}><PictureAsPdfIcon /></IconButton>
185 |   </Tooltip>
186 |   <Tooltip title="Configurar columnas visibles" placement="bottom" arrow>
187 |     <IconButton onClick={(e) => setAnchorEl(e.currentTarget)} sx={{ color: 'white' }}>
188 |       <ViewColumnIcon />
189 |     </IconButton>
190 |   </Tooltip>
191 | </Box>

```

- frontend/src/pages/Reports.tsx:

```

frontend > src > pages > Reports.tsx > Reports > handleGenerarInforme
You, 5 hours ago | 1 author (You)
1 import { Typography, Container, Button, Stack, Tooltip } from "@mui/material";

49 |
50 |     <Tooltip title="Generar tabla de informe con los datos actuales" placement="top" arrow>
51 |       <span>
52 |         <Button
53 |           variant="contained"
54 |           color="primary"
55 |           size="large"
56 |           onClick={handleGenerarInforme}
57 |           disabled={generarInforme}
58 |         >
59 |           INFORME COLECCION
60 |         </Button>
61 |       </span>
    </Tooltip>

```

- frontend/src/pages/ErrorPage.tsx:

```

3 import { useRouteError } from "react-router-dom"; //Hook para obtener el error
4 import { Container, Typography, Button, Stack, Tooltip } from "@mui/material";
5 import { useNavigate } from "react-router-dom";

34 |
35 |     {/* Botón para volver al Login */}
36 |     <Tooltip title="Regresar a la pantalla de Login" placement="bottom" arrow>
37 |       <Button
38 |         variant="contained"
39 |         color="secondary"
40 |         onClick={() => navigate("/")}
41 |       >
42 |         Volver al inicio
43 |       </Button>
    </Tooltip>

```

- frontend/src/components/Menu.tsx:

```

1 import {
2   AppBar,
3   Toolbar,
4   IconButton,
5   Typography,
6   Drawer,
7   Box,
8   List,
9   ListItem,
10  ListItemButton,
11  ListItemIcon,
12  ListItemText,
13  Tooltip
14 } from "@mui/material";

62 |
63 |     {/* Botón hamburguesa para abrir el menú lateral */}
64 |     <Tooltip title="Abrir menú de navegación" placement="right" arrow>
65 |       <IconButton edge="start" color="inherit" onClick={toggleDrawer(true)}>
66 |         <MenuIcon />
67 |       </IconButton>
    </Tooltip>

```

A continuación, la comprobación al ejecutar y pasar el mouse/ratón por encima de cada botón vemos el texto para mejorar la experiencia de usuario e indicar la funcionalidad de cada botón:

- frontend/src/pages/Login.tsx:

Sistema de acceso

Usuario *

Contraseña *

ACCEDER

Iniciar sesión en el sistema

- **frontend/src/pages/Home.tsx:**



- **frontend/src/components/Dashboard.tsx:**

Registro de Productos - SQLite

Complete el formulario para agregar nuevos productos a la base de datos SQLite

Nombre del Producto * Marca *

Tipo * Precio *

Añadir nuevo producto a SQLite

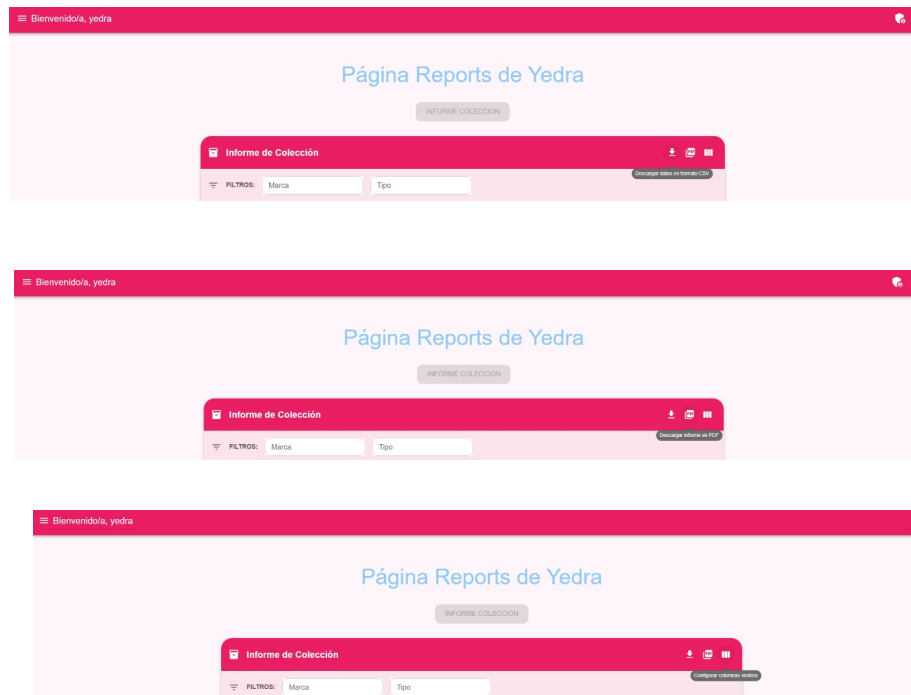
+ INSERTAR DATOS EN SQLITE

Inventario de Productos - SQLite

Acciones	ID	Nombre	Marca	Tipo	Precio
ELIMINAR	12	Altavoz	JBL	Audio	\$35.00
ELIMINAR	8	Teclado	Razer	UTF8	\$20.00
ELIMINAR	7	croissant	editesa	Alimento	\$12.00
ELIMINAR	5	Cargador	Anker	Accesorio	\$12.99
ELIMINAR	3	Ratón	Logitech	Informática	\$15.50
ELIMINAR	2	Auriculares	JBL	Audio	\$25.99

Eliminar este producto permanentemente

- **frontend/src/components/InformeColeccion.tsx:**



- **frontend/src/pages/Reports.tsx:**



- **frontend/src/components/Menu.tsx:**

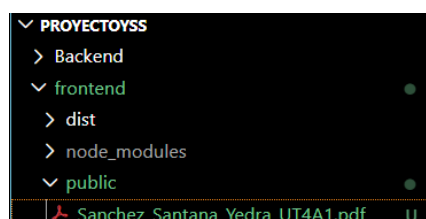


3. Agregar el manual de usuario a tu aplicación

Queremos que cuando el usuario de la aplicación pique en Ayuda se abra una página nueva a parte de la aplicación donde se muestre el PDF del manual de ayuda. Para poder hacer eso tenemos que integrar el manual de ayuda dentro de la aplicación:

a. Mover el archivo PDF:

Vamos a la carpeta de nuestro ordenador y arrastramos el archivo del manual y lo soltamos directamente dentro de la carpeta *frontend/public/* de nuestro proyecto.



b. Modificar frontend/src/components/Menu.tsx:

Abrimos el archivo del menú y hacemos estas dos pequeñas adiciones:

- En la parte superior de los imports, añadimos el icono de ayuda:

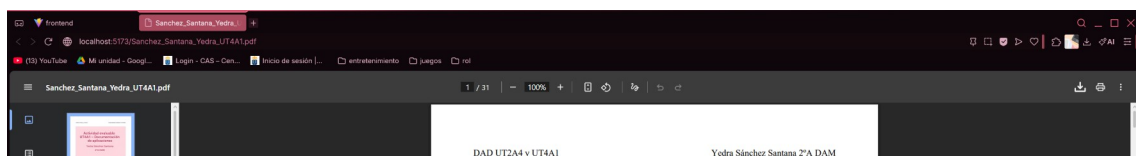
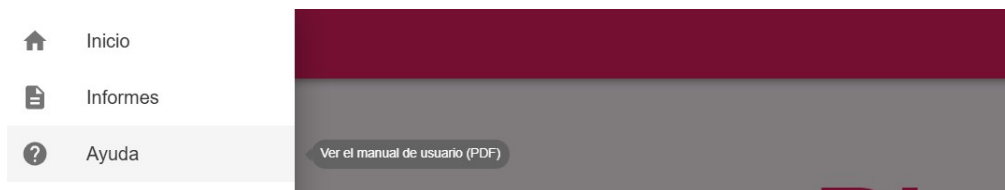
```
16 import HelpIcon from "@mui/icons-material/Help";
```

- Donde tenemos el **Drawer** (menú) añadimos el boton de ayuda:

```
113 { /* Enlace a Ayuda (Abre PDF en pestaña nueva) */ }
114 <ListItem disablePadding>
115   <Tooltip title="Ver el manual de usuario (PDF)" placement="right" arrow>
116     <ListItemButton>
117       component="a"
118       href="/Sanchez_Santana_Yedra_UT4A1.pdf"
119       target="_blank"
120       rel="noopener noreferrer"
121     >
122       <ListItemIcon><HelpIcon /></ListItemIcon>
123       <ListItemText primary="Ayuda" />
124     </ListItemButton>
125   </Tooltip>
126 </ListItem>
127
128 </List>
129 </Box>
130 </Drawer>
```

El target tiene que ser **_blank**. De esta forma le dices que se abra una página nueva a parte. Como lo que se va a abrir al picar en **Ayuda** es un documento PDF, no hace falta añadir el enrutamiento en *frontend/App.tsx*

Comprobación (la url del pdf está en local y el puerto lo que verifica que se ha hecho):



12. Enlace a GitHub

<https://github.com/yedraalumna/ProyectoYSS.git>